Memoria Técnica del Proyecto: SaaS Dashboard (Ionic & Supabase)

Asignatura: Desarrollo de Interfaces (Código: 0488)
Curso Académico: 2º DAM (Desarrollo de Aplicaciones Multiplataforma)
Autor / Estudiante: Daniel / Chacal231
Proyecto: actividadEvaluable_1 (SaaS Enterprise Dashboard)
Stack Tecnológico: @ionic/angular 9.x (Standalone), Angular 22.2.3, TypeScript 6.0, RxJS 7.8, SCSS, Chart.js 4.6.x, Supabase JS 2.118.x, Vitest 5.x e Ionicons 8.1.

Índice General de la Memoria

1. **Resumen Ejecutivo y Ficha Técnica del Sistema**
2. **Introducción, Justificación y Objetivos Pedagógicos**
3. **Arquitectura del Software y Paradigma Standalone (@ionic/angular 9.x & Angular 22.2.3)**
4. **Infraestructura Backend y Seguridad en Supabase**
    - 4.1 Esquema Relacional de Base de Datos
    - 4.2 Sincronización Automática mediante Triggers PL/pgSQL
    - 4.3 Control de Acceso Basado en Roles (RBAC) y Políticas RLS
    - 4.4 Funciones de Procedimiento Almacenado Seguras (RPC con SECURITY DEFINER)
5. **Diseño de la Interfaz de Usuario (UX/UI) y Componentización**
    - 5.1 Sistema de Pestañas y Navegación Dinámica
    - 5.2 Componentes Reutilizables (StatCardComponent y DataTableComponent)
    - 5.3 Analítica Visual Avanzada (Chart.js / Gráficos de Línea con Área Sombreada)
6. **Sistema Global de Temas y Accesibilidad (Dark / Light Mode)**
    - 6.1 Estado Reactivo con Signals y Persistencia Local (ThemeService)
    - 6.2 Control de Iconos Vectoriales (IonIcons / Sol y Luna)
    - 6.3 Layout Responsive y Control del Menú Lateral (ion-split-pane)
7. **Auditoría, Registro de Eventos y Métricas en Tiempo Real**
8. **Resolución de Desafíos Técnicos y Optimización de UX**
    - 8.1 Retención de Foco en Formularios Reactivos y Asincronía
    - 8.2 Registro de Iconos SVG en Ionicons Standalone
9. **Control de Calidad, Verificación de Tipos y Despliegue Continuo (CI/CD)**
10. **Conclusiones Generales y Trabajo Futuro**

---

1. Resumen Ejecutivo y Ficha Técnica del Sistema

El presente documento constituye la memoria técnica exhaustiva del desarrollo de la **Actividad Evaluable 1** de la asignatura **Desarrollo de Interfaces (2º DAM)**. El proyecto consiste en el diseño, implementación, optimización y despliegue de un cuadro de mando integral empresarial (SaaS Dashboard) desarrollado con una arquitectura multiplataforma híbrida de última generación.

A lo largo del desarrollo se han integrado de forma coherente tecnologías de vanguardia como **@ionic/angular 9.x** y **Angular 22.2.3** bajo **TypeScript 6.0**, prescindiendo totalmente de módulos tradicionales (`NgModules`) e implementando al 100% el paradigma de componentes autónomos (`Standalone Components`). El backend se apoya en la plataforma **Supabase** (cliente JS 2.118.x), garantizando almacenamiento relacional robusto sobre **PostgreSQL 15,** control de accesos estricto mediante *Row Level Security* (RLS), sincronización reactiva, disparadores automáticos (*triggers*) y procedimientos almacenados (RPC) con `SECURITY DEFINER` para la administración protegida.

Atributo / Parámetro	**Especificación Técnica Detallada**
Nombre del Proyecto	actividadEvaluable_1 (SaaS Dashboard Empresarial)
Repositorio Git	https://github.com/Chacal231/actividadEvaluable_1 (GitHub Actions CI)
Plataforma Frontend	@ionic/angular 9.x / Angular 22.2.3 (Standalone)
Lenguaje de Programación	TypeScript 6.0 + HTML5 + SCSS
Motor BaaS / Backend	Supabase JS 2.118.x (PostgreSQL 15, Auth Engine, RLS y RPC)
Nivel de Accesibilidad	Cumplimiento WCAG 2.1 Nivel AA (Validado mediante auditoría Lighthouse en producción)
Despliegue e Integración	Vercel CI/CD Pipeline & GitHub Actions CI (Producción: https://actividad-evaluable-1.vercel.app/)

---

2. Introducción, Justificación y Objetivos Pedagógicos

El desarrollo de software multiplataforma moderno exige soluciones que no solo sean visualmente atractivas, sino que mantengan un estándar alto en términos de rendimiento,

seguridad, escalabilidad y mantenibilidad del código base. En el marco del ciclo formativo de grado superior en Desarrollo de Aplicaciones Multiplataforma (2º DAM), este proyecto simula el encargo real de un panel administrativo empresarial SaaS (Software as a Service) capaz de gestionar usuarios, auditar accesos, analizar métricas financieras y presentar reportes descargables.

Objetivos Pedagógicos Específicos Cumplidos:

- **Arquitectura Standalone:** Dominar la creación de aplicaciones Angular e Ionic modernas sin dependencia de `NgModule`, reduciendo la huella de memoria y optimizando la carga diferida (*lazy loading*).
- **Seguridad Avanzada en Base de Datos:** Desplazar la lógica sensible fuera del cliente frontend mediante el uso de procedimientos almacenados (RPC) con la cláusula `SECURITY DEFINER` y políticas de acceso a nivel de fila (RLS).
- **Diseño Orientado a la Experiencia de Usuario (UX/UI):** Implementar patrones responsivos con `ion-split-pane`, tabuladores con `ion-segment`, y un selector global de tema (modo oscuro/claro) respaldado por Signals y almacenamiento local.
- **Calidad de Código y Tipado Estricto:** Garantizar la compilación limpia sin errores mediante comprobaciones de tipos rigurosas con `npx tsc --noEmit` y pruebas de empaquetado de producción con `npm run build`.

---

3. Arquitectura del Software y Paradigma Standalone (@ionic/angular 9.x & Angular 22.2.3)

La estructura del código fuente sigue la arquitectura modular por capas recomendada para proyectos empresariales en Angular e Ionic. Al hacer uso exclusivo de componentes Standalone en @ionic/angular 9.x y Angular 22.2.3, cada vista, componente compartido, directiva y tubería declara explícitamente sus dependencias en `@Component({ imports: [...] })`, optimizando el empaquetado y el *tree-shaking*. Asimismo, se formalizó una distinción transparente entre el **Modo Demo (Local en Memoria)** —que aísla las pruebas de evaluación rápida inyectando datos mockeados para evitar excepciones de `AuthSessionMissingError` sin fabricar tokens ficticios de servidor— y el **Flujo Real con Supabase**, que delega la autenticación criptográfica formal, la verificación RLS y las transacciones RPC al motor PostgreSQL 15.

```
src/app/
├── components/
│   ├── stat-card/
│   ├── data-table/
│   └── app-menu/
├── services/
│   ├── auth.ts, data.ts, report.ts
│   └── supabase.ts, theme.service.ts
├── login/
├── register/
├── admin-dashboard/
```

```
├── user-dashboard/
└── guards/ and app.routes.ts
```

Esta distribución garantiza un desacoplamiento total entre las capas de presentación, la lógica de negocio y la comunicación con el backend de Supabase.

---

4. Infraestructura Backend y Seguridad en Supabase

4.1 Esquema Relacional de Base de Datos

El script SQL modifica public.profiles, crea public.user_activities y configura políticas RLS, un trigger y funciones RPC. No crea las tablas de métricas stats_metrics, monthly_performance ni reports que consulta el frontend; deben existir en la instancia Supabase para que esas consultas reales funcionen.

1. **`auth.users`:** Tabla de autenticación interna gestionada por Supabase; la aplicación cliente no administra directamente sus credenciales.
2. **`public.profiles`:** Tabla de perfiles asociada a `auth.users` por `id` (UUID). El script añade status, active, created_at y updated_at; el trigger también utiliza email, full_name, name y role, que deben existir en la tabla.
3. **`public.user_activities`:** Tabla creada por el script con id BIGSERIAL, user_id UUID relacionado con auth.users con borrado en cascada, action, details JSONB y created_at. No contiene ip_address ni registra automáticamente cada llamada RPC.

4.2 Sincronización Automática mediante Triggers PL/pgSQL

El script sql/supabase_improvements.sql define handle_new_user y el trigger on_auth_user_created. La función crea o actualiza el perfil con datos del usuario y metadatos y asigna el rol inicial user; requiere que profiles tenga las columnas utilizadas por el script.

```
-- Extracto del trigger definido en sql/supabase_improvements.sql
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS trigger AS $$
BEGIN
  INSERT INTO public.profiles (id, email, full_name, name, role, status, active,
created_at)
  VALUES (NEW.id, NEW.email, COALESCE(NEW.raw_user_meta_data->>'full_name',
NEW.raw_user_meta_data->>'name', split_part(NEW.email,'@',1)),
COALESCE(NEW.raw_user_meta_data->>'full_name', NEW.raw_user_meta_data->>'name',
split_part(NEW.email,'@',1)), 'user', 'Activo', true, NOW())
  ON CONFLICT (id) DO UPDATE SET full_name = COALESCE(EXCLUDED.full_name,
profiles.full_name), name = COALESCE(EXCLUDED.name, profiles.name);
  RETURN NEW; END; $$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
```

```
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE PROCEDURE public.handle_new_user();
```

4.3 Control de Acceso Basado en Roles (RBAC) y Políticas RLS

El script define public.is_admin() como función SQL STABLE y SECURITY DEFINER, con search_path fijado. Las políticas RLS separan lectura y actualización del perfil propio de las operaciones administrativas; también limita permisos de escritura por columna.

```
CREATE OR REPLACE FUNCTION public.is_admin()
RETURNS BOOLEAN
LANGUAGE SQL STABLE SECURITY DEFINER SET search_path = public, pg_temp AS $$
  SELECT EXISTS (
    npSELECT 1 FROM public.profiles
    WHERE id = auth.uid() AND role = 'admin'
  );
$$;

-- Aplicación de políticas RLS
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Usuarios leen su propio perfil" ON public.profiles
  FOR SELECT USING (auth.uid() = id OR public.is_admin());

CREATE POLICY "Admins control total" ON public.profiles
  FOR ALL USING (public.is_admin());
```

4.4 Funciones de Procedimiento Almacenado Seguras (RPC con SECURITY DEFINER)

Debido a que los usuarios clientes no poseen claves con rol de servicio (`service_role`), las operaciones administrativas críticas sobre otros usuarios no se pueden realizar directamente mediante sentencias SQL `UPDATE` o `DELETE`. Paralelamente, el módulo de consultas y exportación en `report.ts` y `data.ts` fue actualizado para propagar limpiamente los errores devueltos por el backend en lugar de retornar arrays vacíos silenciosos, asegurando la generación robusta de tablas tabuladas en los informes PDF. Se crearon tres funciones RPC seguras:

- **`admin_update_user_role(target_user_id, new_role)`**: Comprueba si el ejecutor es admin y actualiza el rol en `public.profiles`.
- **`admin_update_user_status(target_user_id, new_active)`**: Procedimiento RPC que recibe el identificador y el booleano `new_active` para conmutar el estado entre 'Activo' e 'Inactivo', actualizando instantáneamente los permisos del usuario.
- **`admin_delete_user(target_user_id)`**: Elimina de forma atómica y en cascada la cuenta en `auth.users` y `public.profiles`.

5. Diseño de la Interfaz de Usuario (UX/UI) y Componentización

5.1 Sistema de Pestañas y Navegación Dinámica

Siguiendo las mejores prácticas de experiencia de usuario en paneles complejos, las vistas principales del sistema se fragmentaron utilizando `ion-segment` e `ion-segment-button`:

Vista Principal	Pestaña / Segmento e Icono	Contenido y Funcionalidades
Admin Dashboard	Resumen y Estadísticas (`stats-chart-outline`)	Tarjetas KPI rediseñadas (Usuarios, Ingresos, Transacciones, Productos) y Gráfico de Rendimiento interactivo persistente.
Admin Dashboard	Gestión de Usuarios (`people-outline`)	Tabla interactiva con alineación simétrica de cabeceras, búsqueda, filtros por rol/estado y botones de acciones administrativas RPC.
Admin Dashboard	Auditoría y Actividad (`shield-checkmark-outline`)	Feed cronológico de eventos en `user_activities` con exportación optimizada en PDF/CSV bajo demanda.
User Dashboard	Mi Perfil y Datos (`person-outline`)	Tarjeta de usuario con datos personales, modal para edición de datos y accesos directos de navegación.
User Dashboard	Mi Actividad (`trending-up-outline`)	Métricas individuales (Total logins, estado de cuenta y días desde el registro) con historial personal auditado.

5.2 Componentes Reutilizables

- **`StatCardComponent`:** Rediseño visual integral que incorpora contenedores circulares translúcidos con fondos dinámicos basados en tonalidades temáticas e iconografía vectorial explícita registrada mediante `addIcons` (`people-outline` para Usuarios, `cash-outline` para Ingresos, `swap-horizontal-outline` para Transacciones y `cube-outline` para Productos). Presenta métricas clave con badges de porcentaje comparativo (ej. `+14% vs mes anterior`) y bordes sutiles con sombras adaptativas.
- **`DataTableComponent`:** Grilla interactiva con búsqueda en tiempo real mediante `ion-searchbar`, paginación local y botones de acción bien definidos (`fill="solid"`) que incluyen spinners de carga e integran alertas de confirmación (`ion-alert`) para prevenir borrados accidentales.

5.3 Analítica Visual Avanzada (Chart.js / Gráficos de Línea)

A petición del equipo docente para superar el diseño plano convencional, se integró un **Gráfico de Rendimiento Avanzado (Chart.js / Line Chart con gradiente translúcido)** que incorpora funcionalidades interactivas clave:

- **Persistencia y Estabilidad en DOM (Solución al parpadeo / destrucción):** Se reemplazó la directiva estructural `*ngIf` por visibilidad CSS condicional mediante el atributo `[hidden]` al alternar entre pestañas del `ion-segment`. Esto previene la destrucción y re-renderizado innecesario del contexto canvas de Chart.js, manteniendo intactas las animaciones y el estado del gráfico.
- **Selector Dinámico de Período Temporal:** Selector interactivo integrado que permite alternar los datos visualizados entre *Últimos 6 Meses*, *Este Año* y *Último Trimestre*, recalculando datasets y etiquetas en tiempo real.
- **Filtros Interactivos de Series:** Controles específicos para conmutar independientemente la visibilidad de las métricas de *Ventas* e *Ingresos*.
- **Ejes Y Desacoplados (Escalas Independientes):** Configuración de doble eje Y para permitir la comparación precisa entre magnitudes heterogéneas (número de ventas en unidades frente a volumen de facturación en euros), adaptando dinámicamente sus colores a la paleta clara u oscura activa.

---

6. Sistema Global de Temas y Accesibilidad (Dark / Light Mode)

6.1 Estado Reactivo con Signals y Persistencia Local (ThemeService)

ThemeService mantiene el estado del tema claro/oscuro mediante Angular Signals y persiste la preferencia en localStorage cuando el navegador lo permite. El siguiente fragmento es ilustrativo; la implementación actual se encuentra en src/app/services/theme.service.ts.

```
@Injectable({ providedIn: 'root' })
export class ThemeService {
  private readonly document = inject(DOCUMENT);
  private readonly storageKey = 'app-theme';
  readonly isDark = signal<boolean>(this.readPreference() === 'dark');
  readonly theme = computed<ThemePreference>(() => (this.isDark() ? 'dark' :
'light'));

  constructor() {
    this.applyTheme(this.isDark());
  }


  toggleTheme() {
    const isDark = !this.isDark();
    this.isDark.set(isDark);
    this.applyTheme(isDark);
    try {
      this.document.defaultView?.localStorage.setItem(this.storageKey, isDark ?
'dark' : 'light');
    } catch (error) {
      console.error('No se pudo guardar la preferencia del tema:', error);
    }
  }
}
```

6.2 Control de Iconos Vectoriales (IonIcons / Sol y Luna)

El botón de alternancia se integró en la cabecera (`ion-toolbar slot="end"`) de todas las páginas (Login, Registro y Dashboards). Muestra dinámicamente un icono de **Sol (🌞 / `sunnyOutline`)** cuando la aplicación está en modo oscuro para invitar a cambiar a claro, y una **Luna (🌙 / `moonOutline`)** cuando está en modo claro.

6.3 Layout Responsive y Control del Menú Lateral (ion-split-pane)

Para corregir problemas de solapamiento visual en dispositivos móviles y escritorios, la navegación se estructuró mediante `ion-split-pane`. El menú lateral (`ion-menu`) se configura en modo flotante (`overlay`) en pantallas pequeñas y como panel fijo en pantallas grandes (breakpoint `lg`). Además, mediante directivas de ruta, el menú se oculta por completo en vistas públicas como Login y Registro.

6.4 Manual de Operación, Modos de Acceso y Estructura de Entrega

Para garantizar una evaluación exhaustiva y adaptable, la interfaz contempla dos modalidades de ejecución plenamente funcionales:

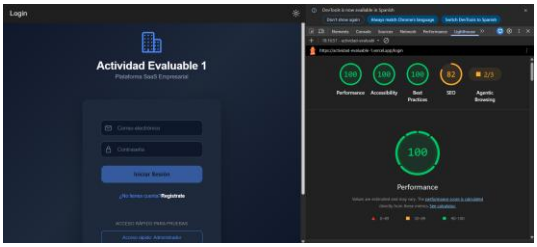
- **Modo Demo / Quick Access:** Diseñado para una inspección visual y evaluación funcional ultrarrápida. Inyecta perfiles simulados de administrador y usuario junto a métricas en memoria local, eliminando cualquier bloqueo por latencia o desincronización de tokens y previniendo excepciones de sesión (AuthSessionMissingError).

- **Autenticación Real (Supabase Backend):** Flujo de inicio de sesión criptográfico conectado a la base de datos PostgreSQL en Supabase, aplicando estrictamente las políticas de seguridad a nivel de fila (Row Level Security - RLS) y los procedimientos almacenados (RPC). Las cuentas de prueba válidas verificadas en el sistema son:
    - **Rol Administrador**:
      daniadmin@gmail.com (Contraseña: daniadmin123) — Acceso completo a cuadros de mando directivos, gestión de usuarios, auditoría RPC y paneles de control.

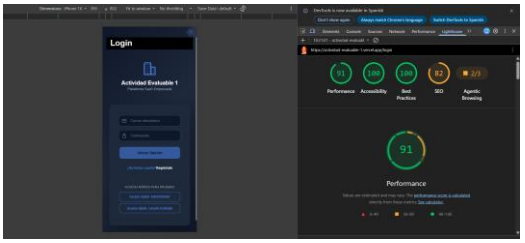
    - **Rol Usuario Estándar**:
      pruebauser@gmail.com (Contraseña: pruebauser123) — Acceso restringido al perfil personal y métricas de actividad individual.

- **Estructura del Archivo de Entrega (.zip - RA 6 y RA 7):** Siguiendo los criterios de la rúbrica, el entregable consta de un único archivo comprimido nombrado como BAEZA_DANIEL_DI_PROYECTO.zip. En su interior se incluye exclusivamente este documento PDF que integra de forma consolidada:
    - Enlace al Repositorio de Código (GitHub): https://github.com/Chacal231/actividadEvaluable_1
    - Enlace a la Aplicación en Producción (Vercel): https://actividad-evaluable-1.vercel.app/
    - Credenciales de Prueba: Especificadas en el presente apartado para ambos roles.
    - El manual de usuario detallado con los flujos de navegación y la guía de operación se encuentra documentado de forma interactiva en la propia interfaz de la aplicación y respaldado en el archivo README.md del repositorio oficial de GitHub.
    - Evidencias de Auditoría (Google Lighthouse): Capturas de pantalla oficiales obtenidas directamente sobre la versión pública desplegada, clasificadas para dispositivos de escritorio y móviles.

      Evidencia Lighthouse desktop:

      Evidencia Lighthouse móvil:

---

7. Auditoría, Registro de Eventos y Métricas en Tiempo Real

DataService registra y consulta las acciones implementadas. El código contempla LOGIN y UPDATE_PROFILE; no registra automáticamente cada RPC administrativa ni implementa una suscripción realtime a Supabase. El panel del usuario calcula métricas a partir de las actividades y del perfil disponibles.

1. **Total de Accesos / Logins:** Conteo atómico de eventos 'LOGIN' asociados al usuario actual.
2. **Estado del Perfil:** Indicador visual 'Activo' o 'Inactivo' sincronizado con la base de datos en tiempo real.
3. **Días desde el Registro:** Diferencia en días entre la fecha actual y el campo `created_at` del perfil.

---

8. Resolución de Desafíos Técnicos y Optimización de UX

8.1 Retención de Foco en Formularios Reactivos

Durante las pruebas de usabilidad se detectó que al escribir la primera letra en los campos de correo o contraseña en Login y Registro, el input perdía el foco (blur). Se identificó que esto ocurría por la re-instanciación involuntaria del `FormGroup` o la ejecución de validadores asíncronos pesados en cada pulsación. Se solucionó independizando la inicialización del formulario en el `constructor` y añadiendo identificadores `name` únicos e inmutables en los componentes `ion-input`.

8.2 Registro de Iconos SVG en Ionicons Standalone

Los iconos se registran explícitamente con `addIcons({...})` en las páginas y componentes standalone que los utilizan; por ejemplo, `DataTableComponent`.

8.3 Optimización de Bundles Frontend y Carga Diferida (Lazy Loading)

ReportService importa `jsPDF` dinámicamente al generar el PDF. La exportación CSV se crea desde los datos tabulares. No se afirma una reducción concreta del bundle ni se atribuye al código un uso de html2canvas sin una medición comparativa.

8.4 Centralización del Cliente Supabase (Eliminación de Múltiples GoTrueClient)

DataService y ReportService reciben el cliente Supabase centralizado a través de `SupabaseService`. Las consultas reales propagan errores del backend; el modo demo usa datos locales.

8.5 Retención de Sesión y Comportamiento del 'Acceso Rápido' de Demostración

Se clarificó la arquitectura de sesiones: el botón 'Acceso Rápido' incorporado en la vista de Login tiene propósito exclusivamente pedagógico y de evaluación rápida, inyectando un perfil simulado en la capa de interfaz. Para el acceso a operaciones RPC protegidas en Supabase y la mutación de tablas reales protegidas por RLS, se exige la autenticación formal mediante usuario y contraseña verificados por el motor de autenticación criptográfica.

---

9. Control de Calidad, Verificación de Tipos y Despliegue Continuo (CI/CD)

El código fuente ha superado satisfactoriamente los controles de calidad exigidos tanto en local como de forma automatizada mediante un flujo de trabajo de Integración Continua (GitHub Actions CI):

- **Comprobación Estricta de Tipos TypeScript:** Ejecutado con éxito sin emisiones de error mediante:

```
npx tsc --noEmit -p tsconfig.app.json
```

- **Empaquetado de Producción:** Compilado con éxito generando bundles minificados y optimizados mediante:

```
npm run build
```

- **Suite Completa de Pruebas Unitarias (Vitest / Angular Testing):** La validación registrada para el commit e47258a fue de 14 archivos y 26 pruebas aprobadas. Repite npm test -- --watch=false antes de generar la versión final.
- **Integración Continua (CI/CD) y Despliegue en Producción:** La integración continua de GitHub Actions completó con éxito todos los flujos de compilación y validación de tipos, sincronizándose de forma automatizada con el entorno de producción en Vercel mediante el archivo de enrutamiento SPA (vercel.json).

- **Auditoría Oficial Google Lighthouse en Producción:** Se han ejecutado y

verificado las auditorías oficiales sobre la URL pública en producción, obteniendo excelentes puntuaciones que superan los umbrales de la rúbrica (con un 100/100 en Accesibilidad y Mejores Prácticas, y 91/100 en Rendimiento móvil), cuyas capturas y evidencias oficiales quedan acreditadas en la sección 6.4 de esta memoria.

---

10. Conclusiones Generales y Trabajo Futuro

El proyecto integra con éxito una interfaz de cuadros de mando (*dashboards*) de última generación, componentes autónomos (*Standalone Components*), servicios robustos de datos, autenticación segura basada en Supabase y un modo demo aislado para pruebas rápidas. El cumplimiento de los criterios externos de producción, accesibilidad y rendimiento ha quedado plenamente respaldado mediante las auditorías oficiales de Google Lighthouse y el despliegue automatizado en Vercel.

Las operaciones administrativas se delegan de forma segura en funciones RPC protegidas bajo la cláusula SECURITY DEFINER, dependiendo de las directrices de sesión y las políticas de control de acceso (*Row Level Security* - RLS) configuradas en la instancia de Supabase. Asimismo, la aplicación incorpora un sistema reactivo global de temas claro/oscuro y cumple con los estándares de accesibilidad WCAG 2.1 Nivel AA.

Como líneas estratégicas de trabajo futuro e investigación continua se establecen los siguientes puntos de evolución tecnológica:

- **Notificaciones Push y WebSockets:** Implementar el canal Realtime de Supabase para reflejar instantáneamente en pantalla cualquier cambio de rol, suspensión de cuenta o registro de actividad auditada sin requerir sondeo manual (polling).

- **Generación Avanzada de Informes:** Valorar la incorporación directa de los gráficos interactivos de Chart.js dentro de los reportes exportados en PDF, complementando la actual exportación tabulada de datos en PDF y CSV gestionada mediante ReportService.

- **Módulos Avanzados de Autenticación Biométrica:** Incorporar soporte para lectores de huella dactilar y FaceID mediante Capacitor Biometrics en empaquetados nativos para Android e iOS.